# Step 3 — The Pipeline — Solution

> Back to the step · Hint 1 · Hint 2

**Story:** `ask` ties it together — read the schema, generate the SQL, **guard it**, run it, and summarize the rows — plus `main.py` and the integration test. **If not:** the tested pieces never meet, and there's no single call that answers a plain-English question safely.

## The pipeline

```
# src/pipeline.py
# Read the schema, generate a SELECT, guard it, run it, and summarize the
# rows. The guard sits between generation and execution - nothing unsafe
# ever reaches the database.
from src.answer import summarize_answer
from src.db.runner import run_select
from src.db.schema import schema_text
from src.exceptions import UnsafeQueryError
from src.guard import is_safe_select
from src.sql_agent import generate_sql
from src.utils.logger import get_logger


def ask(model, db_path, question):
    logger = get_logger(__name__)
    schema = schema_text(db_path)
    sql = generate_sql(model, schema, question)
    if not is_safe_select(sql):
        raise UnsafeQueryError(f"Refused unsafe query: {sql}")
    logger.info(f"running: {sql}")
    rows = run_select(db_path, sql)
    answer = summarize_answer(model, question, rows)
    return answer, sql, rows
```

## main.py

```
# main.py
import sys
```

```
from src.config import load_config
from src.db.seed import build_sample_db
from src.exceptions import UnsafeQueryError
from src.models.llm import get_model
from src.pipeline import ask


def main():
    if len(sys.argv) < 2:
        print('usage: python main.py "your question"')
        return
    question = sys.argv[1]

    config = load_config()
    build_sample_db(config.db_path)
    model = get_model(config)

    try:
        answer, sql, rows = ask(model, config.db_path, question)
    except UnsafeQueryError as error:
        print(f"\nRefused: {error}")
        return

    print(f"\nSQL:\n  {sql}")
    print(f"\nAnswer:\n  {answer}")


if __name__ == "__main__":
    main()
```

## The integration test (real model)

```
# tests/integration/test_live_ask.py
# Real model + SQLite. Run from the repo root:
#   python -m tests.integration.test_live_ask
from src.config import load_config
from src.db.seed import build_sample_db
from src.models.llm import get_model
from src.pipeline import ask


def test_counts_completed_orders():
    config = load_config()
    build_sample_db(config.db_path)
```

```
    model = get_model(config)
    answer, sql, rows = ask(
        model, config.db_path, "How many orders are completed?")
    ok = "select" in sql.lower() and "3" in answer
    status = "OK" if ok else "WRONG"
    print(f"case 1 (completed orders counted) -> {status}")


def test_lists_a_city():
    config = load_config()
    build_sample_db(config.db_path)
    model = get_model(config)
    answer, sql, rows = ask(
        model, config.db_path, "Which city is Ada in?")
    ok = "london" in answer.lower()
    status = "OK" if ok else "WRONG"
    print(f"case 2 (city answered)            -> {status}")


if __name__ == "__main__":
    test_counts_completed_orders()
    test_lists_a_city()
```

**Expected run:**

```
$ python main.py "How many orders are completed?"

SQL:
  SELECT count(*) FROM orders WHERE status = 'completed'

Answer:
  There are 3 completed orders.
```

That's the whole project: the question turned into a query, the query checked by a deterministic guard before it runs, and the returned rows phrased as one plain sentence — so a non-technical user gets a real answer from the database, and no query that isn't a single read-only `SELECT` ever reaches it.